

SepsisScope

Retinal Fundus AVR Pipeline Technical Documentation — v1.0

This document describes the complete SepsisScope image-processing pipeline: its architecture, algorithms, assumptions, edge-case handling, known corrections, and final outputs.

Generated: 2026-05-18 | Datasets: DRIVE · STARE · ARIA | Language: Python 3

1. Overview

SepsisScope is a fully automated retinal fundus image analysis pipeline that computes the **Arteriovenous Ratio (AVR)** — a non-invasive biomarker for microvascular health. Low AVR values are associated with arteriolar narrowing seen in hypertension, cardiovascular disease, and sepsis-related microvascular injury. High AVR values indicate venular dilation linked to inflammation and metabolic syndrome.

The pipeline accepts raw colour fundus photographs from standard retinal cameras and produces a numeric AVR per image through five sequential, independently runnable processing steps.

Supported Datasets

Dataset	Format	Typical resolution	Notes
DRIVE	.tif	565 × 584 px	20 training + 20 test images
STARE	.ppm	700 × 605 px	Mixed pathology
ARIA	.tif	768 × 576 px	Diabetic / AMD / control groups

Technology Stack

Library	Purpose
OpenCV	Image I/O, CLAHE, morphology, distance transform, drawing
scikit-image	Frangi filter, skeletonisation, connected-component labelling
NumPy / SciPy	Array operations, convolution for branch-point detection
Python csv	All intermediate data stored as plain CSV — no database

2. Pipeline Architecture

Each step is a self-contained script that reads its inputs from disk and writes outputs to a specified folder. Steps run sequentially; each step depends only on the outputs of the preceding one.

```
Fundus Image
  ■
  ▼
[step_green_channel.py] ← Exploratory channel analysis
  ■
  ▼
[Step 1] Vessel Segmentation + Skeletonisation
  ■ → binary mask · skeleton · per-segment CSV
  ▼
[Step 2] Optic Disc Detection
  ■ → disc centre (cx, cy) and radius r
```

```
▼  
[Step 3] Zone Filtering (1r - 2r annular zone)  
■ → filtered segments in measurement zone  
▼  
[Step 4] Arteriole / Venule Classification  
■ → each segment labelled A or V  
▼  
[Step 5] CRAE / CRVE / AVR Calculation  
→ Arteriovenous Ratio per image
```

3. Step 0 — Green Channel Exploration

File: step_green_channel.py

Purpose

An exploratory/diagnostic script that splits a single DRIVE training image into its RGB channels and displays them side by side. This step is not part of the automated batch pipeline — it exists to visually justify a foundational design choice.

Key Finding

The **green channel** provides the highest vessel-to-background contrast in fundus photography:

- Red channel: saturates near the bright optic disc; vessels poorly separated.
- Blue channel: high noise floor due to short-wavelength scattering.
- Green channel: vessels appear as dark ridges on a bright retinal background — ideal for ridge-detection filters.

Note: This channel choice propagates to every downstream step. All vesselness, brightness, and disc detection computations use the green channel.

4. Step 1 — Vessel Segmentation + Skeletonisation

File: `step1_segment_skeleton.py`

Goal

Detect and map every blood vessel in a fundus image, reduce each vessel to its single-pixel centreline (skeleton), and extract per-segment geometric and photometric measurements for downstream analysis.

Processing Pipeline (per image)

Sub-step	Method / Detail
Load	<code>cv2.imread</code> → BGR → RGB conversion
Green channel	<code>img_rgb[:, :, 1]</code> , normalised to [0, 1]
Retinal mask	Largest bright contour → min-enclosing circle → erode 15 px inward
Contrast enhance	CLAHE (<code>clipLimit=2.0</code> , <code>tileGridSize=8×8</code>) → re-apply mask
Vesselness	Frangi filter: sigmas 1–5, <code>black_ridges=True</code> , $\alpha=\beta=0.5$, $\gamma=15$
Binarisation	Threshold at 0.02 on rescaled vesselness map
Morphological clean	MORPH_OPEN with 3×3 elliptical kernel — removes speckle noise
Skeletonisation	<code>skimage.morphology.skeletonize</code> → single-pixel centrelines
Branch detection	3×3 convolution counts 8-connected neighbours; ≥ 3 = branch point
Segment labelling	<code>measure.label</code> (<code>connectivity=2</code>) — each disconnected piece = one segment

Per-Segment Measurements

Feature	How computed
<code>length_px</code>	Number of skeleton pixels in segment
<code>width_px</code>	$2 \times \text{mean}(\text{distance_transform at skeleton pixels})$
<code>r_mean / g_mean / b_mean</code>	Mean RGB at skeleton pixel locations in original image
<code>color_score</code>	$r_mean / (g_mean + b_mean + \epsilon)$ — arterioles score higher
<code>contrast</code>	$ g_mean - \text{local_background_mean} $ sampled in 5 px radius
<code>centroid_x / y</code>	From <code>skimage.regionprops</code>

Assumptions

- Vessels are **dark ridges** on a bright background in the green channel (`black_ridges=True`). Valid for standard fundus cameras; may fail for non-mydriatic or infrared images.

- Frangi sigma range [1, 5] captures capillaries through medium vessels. Use [1, 8] for high-quality final runs.
- Width via distance transform is a proxy for true vessel calibre (no calibration to physical microns).
- Segments shorter than min_area=30 px are noise and discarded.
- Local background is sampled at ≤ 20 evenly-spaced points per segment for speed.

Corrections / Edge Cases

The hard circular boundary of the retinal disc can appear as a strong ridge to the Frangi filter, producing false vessel detections at the image border. This is corrected by:

- Eroding the retinal mask 15 px inward *before* Frangi processes the image.
- Re-applying the mask after CLAHE to suppress CLAHE artefacts at the edge.
- Falling back to image centre if contour detection fails (very dark images).

Outputs

File	Description
<stem>_mask.png	Binary vessel mask
<stem>_skeleton.png	Single-pixel centreline image
<stem>_overlay.png	Colour overlay: vessels=green, branch points=red
<stem>_segments.csv	One row per segment with all measurements
step1_summary.csv	Batch summary: n_segments, coverage% per image

5. Step 2 — Optic Disc Detection

File: step2_disc_detect.py

Goal

Locate the optic disc centre (cx, cy) and estimate its radius r . This is the anatomical reference point for the $1r-2r$ measurement annulus required by the Knudtson AVR protocol.

Three-Tier Detection Cascade

Three methods are tried in sequence; the first to exceed the confidence threshold (0.25) is used:

Method 1 — Vessel Convergence (Primary)

All major retinal vessels radiate from the optic disc. The skeleton from Step 1 is blurred with a large Gaussian (kernel 151×151 , $\sigma=40$) to create a vessel density heatmap. The peak of this map is taken as the disc centre.

Confidence = $\text{peak_density} / (5 \times \text{mean_density})$, clamped to $[0, 1]$.

Method 2 — Brightness-Guided (Fallback, $\text{conf} < 0.25$)

The optic disc is the brightest compact region in the fundus image. The green channel is blurred (61×61 , $\sigma=20$) and the image centre is suppressed (within 20% of min-dimension radius) to avoid mistaking it for the disc. The brightest remaining point is the disc centre.

Method 3 — Positional Prior (Last Resort, always flagged)

The disc lies in the left or right third of the image. The side with higher mean green intensity is chosen; the centre is placed at its horizontal midpoint and the image vertical midpoint. Confidence = 0.0. Flag = LOW_CONFIDENCE_FALLBACK.

Radius Estimation

Radial brightness is sampled at 16 points on circles of increasing radius. The disc edge is defined where mean ring brightness drops below 45% of centre brightness. Radius is capped at 15% of image min-dimension. Default fallback = 8% of image min-dimension.

Assumptions

- Disc = highest local vessel density in the image.
- Disc radius $\approx$ 8% of image size — consistent across DRIVE/STARE/ARIA.
- Images are roughly horizontally oriented (disc in left or right third). Fails for rotated or vertically-framed images.

Outputs

File	Description
<stem>_disc.png	Overlay: yellow circle=disc, red dot=centre, blue rings= $1r/2r$
step2_disc_results.csv	disc_cx, disc_cy, disc_r, confidence, method, flag per image

6. Step 3 — Zone Filtering (1r – 2r)

File: `step3_zone_filter.py`

Goal

Retain only vessel segments that lie in the **1r to 2r annular zone** — the standard clinical measurement annulus used by ophthalmologists to assess CRAE, CRVE, and AVR per the Knudtson 2003 protocol.

Dual Inclusion Criterion

A segment is **kept** if **either** condition is met:

- **Centroid in zone** — centroid distance from disc centre lies in $[r, 2r]$.
- **Pixel majority** — $\geq 30\%$ of skeleton pixels fall within the annular zone.

Using OR instead of AND increases recall, correctly accepting segments that straddle the zone boundary rather than discarding them arbitrarily.

Merge with Step 1 Measurements

Step 3 re-labels segments independently from Step 1, so IDs don't match. Centroids are matched spatially with a **55 px tolerance** to retrieve the original `width_px`, `color_score`, `contrast`, and `r/g/b_mean` from the Step 1 CSV.

Assumptions

- Low-confidence disc detections (`flag=LOW_CONFIDENCE_FALLBACK`) are skipped entirely — an unreliable disc centre would incorrectly filter all segments.
- 55 px centroid tolerance is intentionally generous because re-labelling can shift segment centroids by several pixels.
- Skeleton fragments shorter than 5 px are noise and discarded.

Known Issue — Dead Code

Note: `filter_segments_by_zone()` contains an unreachable duplicate implementation after its return statement (lines 116–173 of `step3_zone_filter.py`). This is a remnant of an earlier refactor and has no runtime impact, but should be removed to avoid confusion.

Outputs

File	Description
<code><stem>_zone.png</code>	Overlay: kept=green, rejected=red, 1r/2r rings=blue
<code><stem>_filtered.csv</code>	Zone-passing segments with Step 1 measurements merged

step3_summary.csv	Per-image: n_kept, n_rejected, disc_r, disc_conf
-------------------	--

7. Step 4 — Arteriole / Venule Classification

File: `step4_classify.py`

Goal

Label each zone-filtered vessel segment as **arteriole (A)**, **venule (V)**, or **uncertain**. This classification feeds the Knudtson width-combination formula in Step 5.

Classification Algorithm (per image)

1. **Normalise** `color_score` and `width_px` to `[0, 1]` range within that image.
2. **Combined score** = `norm_color` – `norm_width`

Sign of combined score	Interpretation
Positive (redder AND narrower)	Arteriole
Negative (darker AND wider)	Venule

3. **Label assignment:**

- `|combined_score| < 0.05` → uncertain
 - `combined_score > median(combined)` → arteriole
 - else → venule
4. Sort all results by confidence (= `|combined_score|`) descending.

Assumptions

- Arterioles are **narrower and redder** than venules in the same image. This is a well-established clinical fact in retinal imaging.
- Per-image normalisation makes the classifier dataset-agnostic — no cross-image calibration or training data required.
- Median split as the decision boundary assumes a roughly equal mix of A and V per image, which holds for normal retinae.
- Segments with `color_score` ≤ 0 or `width_px` ≤ 0 are invalid and discarded.

Known Limitations

- Entirely rule-based — no ground-truth training. On pathological images (silver-wiring, AV nicking), the A/V colour difference may be reduced, lowering classification accuracy.
- Segment IDs in the overlay image may not perfectly align with classified IDs because the skeleton used for overlay (from Step 1) differs from the re-labelled skeleton in Step 3.

Outputs

File	Description
<stem>_classified.csv	Segments with label, combined_score, confidence
<stem>_classified.png	Overlay: arterioles=red, venules=blue, uncertain=yellow
step4_summary.csv	Per-image: n_arteriole, n_venule, n_uncertain, n_total

8. Step 5 — CRAE / CRVE / AVR Calculation

File: `step5_avr.py`

Goal

Compute the **Central Retinal Arteriole Equivalent (CRAE)**, **Central Retinal Venule Equivalent (CRVE)**, and their ratio **AVR = CRAE / CRVE** — the primary clinical output of the pipeline.

Knudtson-Modified Hubbard Formula

Reference: Knudtson et al. (2003) *Ophthalmology* 110(8):1491–1496

Protocol

- Take the **6 widest arterioles** and **6 widest venules** by `width_px`.
- Apply iteratively, combining pairs widest-first:

```
Arteriole:  W = 0.88 × √(w12 + w22)
Venule:      W = 0.95 × √(v12 + v22)

Repeat until one value remains per type.
AVR = CRAE / CRVE
```

AVR Interpretation

AVR Range	Flag	Clinical Interpretation
0.6 – 0.8	normal	Normal microvascular calibre ratio
< 0.6	low_avr	Arteriolar narrowing — hypertension, cardiovascular risk
> 0.8	high_avr	Venular dilation — inflammation, metabolic syndrome

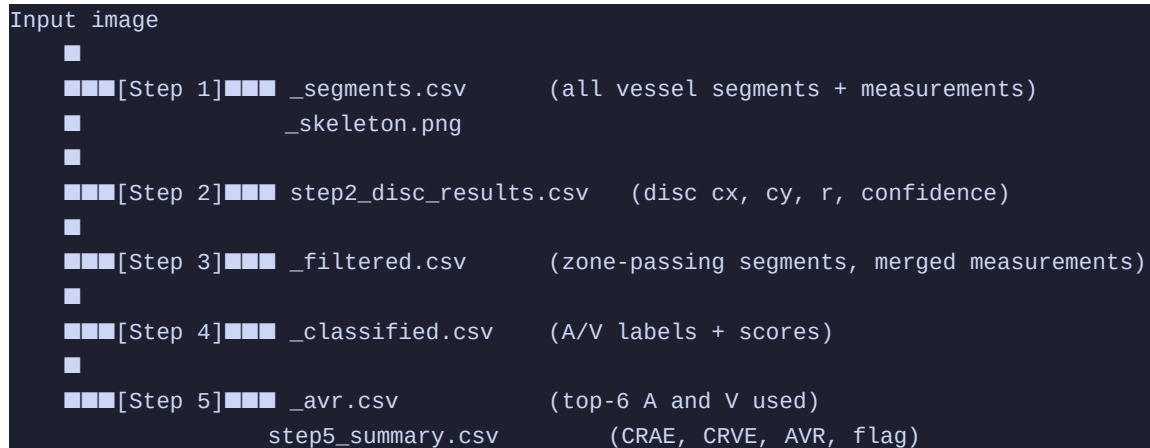
Assumptions

- Vessel width in **pixels** is used directly (not converted to microns). CRAE/CRVE values are therefore image-resolution-dependent and not directly comparable across datasets with different camera fields of view.
- Minimum 6 vessels of each type required. Images with fewer are skipped.
- Top-6 selection by width matches the Knudtson protocol's intent to use the largest, most clinically significant vessels.
- Width from Step 1 distance transform is a reasonable proxy for true vessel diameter under the symmetric cross-section assumption.

Outputs

File	Description
<stem>_avr.csv	12 selected vessels (6A + 6V) with selected_for and rank_width
step5_summary.csv	CRAE, CRVE, AVR, flag per image + aggregate stats (min/max/mean/std, % in each range)

9. End-to-End Data Flow



10. Global Assumptions

#	Assumption	Scope
1	Green channel gives best vessel contrast	All steps
2	Pixel width $\approx$ vessel calibre (no micron conversion)	Steps 1, 5
3	Vessels are dark ridges on bright background (green ch)	Step 1
4	Disc is in left or right third of image	Step 2 fallback
5	Roughly equal mix of A and V per image	Step 4
6	Top-6 widest vessels best represent CRAE/CRVE	Step 5

11. Known Issues & Corrections Needed

Location	Issue	Severity
step3_zone_filter.py:116–173	Dead code after return — unreachable duplicate implementation	Low
Step 4 overlay	Segment IDs from re-labelling may not match skeleton — wrong Medial colours	Medium
Step 5	Width in pixels not microns — CRAE/CRVE not comparable cross-study	Medium
Step 2 fallback	Positional prior assumes horizontal orientation — fails for rotated images	Low
Step 1 width	Distance transform $\times 2$ assumes symmetric vessel cross-section	Low

12. Usage Quick Reference

Step 1 — Segmentation

```
python step1_segment_skeleton.py \  
  --input C:/SepsisScope/data/DRIVE \  
  --output C:/SepsisScope/out/step1 \  
  --sigmas 1 8 --threshold 0.02 --workers 4
```

Step 2 — Disc Detection

```
python step2_disc_detect.py \  
  --images C:/SepsisScope/data/DRIVE \  
  --step1_out C:/SepsisScope/out/step1 \  
  --output C:/SepsisScope/out/step2
```

Step 3 — Zone Filtering

```
python step3_zone_filter.py \  
  --images C:/SepsisScope/data/DRIVE \  
  --step1_out C:/SepsisScope/out/step1 \  
  --step2_csv C:/SepsisScope/out/step2/step2_disc_results.csv \  
  --output C:/SepsisScope/out/step3
```

Step 4 — Classification

```
python step4_classify.py \  
  --step3_drive C:/SepsisScope/out/step3 \  
  --step1_drive C:/SepsisScope/out/step1 \  
  --output_drive C:/SepsisScope/out/step4_drive \  
  --dataset drive
```

Step 5 — AVR Calculation

```
python step5_avr.py \  
  --step4_drive C:/SepsisScope/out/step4_drive \  
  --output_drive C:/SepsisScope/out/step5_drive \  
  --dataset drive
```